MercaUJAP

PROYECTO FINAL · LPR07304 LENGUAJES DE PROGRAMACIÓN · UJAP

Guía del proyecto

Qué es el sistema, cómo ejecutarlo, cómo funciona por dentro y cómo presentarlo. Pensada para entenderla aunque sepas poco de programación.

Python · FastAPI | Node.js · Express | Java · Spring Boot | Go · Gin | PostgreSQL | MongoDB | React · shadcn/ui | Docker | Nginx

REPOSITORIO
github.com/stvnalc/mercaujap-microservicios

PARA EJECUTARLO
docker compose up --build

ABRIR EN EL NAVEGADOR
http://localhost:8080

USUARIO DE PRUEBA
admin@mercaujap.com · admin123

INTEGRANTES
Steven Alcalá · Jorge Fattal · Eyleen Berrio · Ricardo Elbazi

Octubre de 2026

Contenido

1 Qué es MercaUJAP — en palabras simples

2 Cómo ejecutarlo paso a paso — Windows y Mac

3 Recorrido por el sistema — con capturas

4 Cómo está construido — microservicios y bases de datos

5 El viaje de un pedido — qué pasa por dentro

6 Tiempo real, pago y envío — WebSocket y datos ordenados

7 Inicio de sesión y permisos

8 Conceptos que necesitas saber

9 Dónde está cada cosa — carpetas y archivos

10 Guion para la presentación — y rúbrica

11 Preguntas que puede hacer la profesora

12 Problemas comunes y soluciones

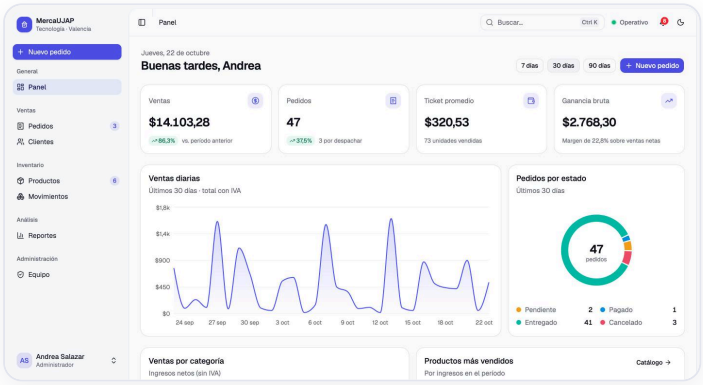
¿Cómo leer esta guía?

Si solo quieres **verlo funcionando**, ve a la sección 2. Si vas a **presentarlo**, lee también las secciones 4, 5, 6, 10 y 11. Los términos técnicos (API, contenedor, WebSocket…) se explican en la sección 8.

1 Qué es MercaUJAP

Es el **panel de gestión de una tienda de tecnología**: lo que usa el personal (administradores y vendedores) para manejar el negocio día a día.

- **Pedidos:** registrar las ventas y llevarlas de Pendiente a Pagado, Enviado y Entregado (o Cancelado). Guarda el **banco** y el **número de referencia** del pago, la **empresa de envío** y el **número de guía**. Calcula el IVA (16 %) e imprime la nota de entrega.
- **Trabajo en equipo en tiempo real:** si un compañero hace un cambio, tu pantalla se actualiza sola y te avisa quién lo hizo.
- **Productos e inventario:** precios, costos, cuánto se gana con cada producto y cuántas unidades hay. Cada entrada o salida de mercancía queda anotada (*kardex*).
- **Clientes:** datos de contacto, cédula/RIF e historial de compras.
- **Panel y reportes:** ventas, ganancia, productos más vendidos, gráficos, y reportes que se descargan para Excel.
- **Seguridad:** hay que iniciar sesión, y cada rol tiene permisos distintos.



Panel principal: indicadores del mes, ventas por día y pedidos por estado.

La idea principal: microservicios (analogía del restaurante)

En vez de **un solo programa gigante** que hace todo (un *monolito*), el sistema está dividido en **4 programas pequeños** llamados **microservicios**. Cada uno hace una sola cosa, está escrito en un **lenguaje distinto** y tiene su **propia base de datos**.

EN UN RESTAURANTE…	EN MERCAUJAP	LENGUAJE
La recepción: lista de clientes y quién es empleado	**Clientes y autenticación**	Python
El almacén, con su libreta de entradas y salidas	**Catálogo e inventario**	Node.js
El mesero que toma y sigue cada pedido	**Pedidos**	Java
El contador que saca las cuentas	**Reportes**	Go
El portero que revisa tu carnet y te manda a la mesa correcta	**API Gateway**	Nginx
La pantalla que ve el personal	**Frontend** (página web)	React + shadcn/ui

Cuando el **mesero (Java)** toma un pedido, le pregunta a la **recepción (Python)** si el cliente existe y al **almacén (Node)** cuánto cuesta cada cosa y si hay suficiente. Si todo está bien, el almacén descuenta lo vendido. Eso es la **comunicación entre microservicios**.

2 Cómo ejecutarlo paso a paso

Solo necesitas **Docker Desktop**. No hay que instalar Python, Java, Node ni Go: cada parte del sistema trae lo que necesita dentro de su contenedor.

1. **Instala Docker Desktop** desde `docker.com/products/docker-desktop`. En Windows deja marcada la opción **WSL 2** (puede pedir reiniciar). Ábrelo y espera a que diga *"Engine running"*.

2. **Descarga el proyecto.** Con Git:

```
git clone https://github.com/stvnalc/mercaujap-microservicios.git
cd mercaujap-microservicios
```

   Sin Git: en GitHub pulsa **Code** → **Download ZIP**, descomprímelo y abre una terminal dentro de la carpeta.
   En Windows: abre la carpeta en el Explorador, escribe `cmd` en la barra de direcciones y pulsa Enter.

3. **Levanta todo** con un solo comando:

```
docker compose up --build
```

   La primera vez tarda entre 3 y 10 minutos (descarga y compila todo). Está listo cuando aparece `[java] Datos demo: 84 pedidos de ejemplo creados`.

4. **Abre** `http://localhost:8080` en el navegador e **inicia sesión** (en la pantalla de entrada hay botones que llenan los datos solos):

ROL	CORREO	CONTRASEÑA
Administrador	`admin@mercaujap.com`	`admin123`
Vendedor	`vendedor@mercaujap.com`	`vendedor123`

5. **Para apagarlo:** `Ctrl + C` en la terminal, o `docker compose down`.

> **Datos de ejemplo**
>
> El sistema arranca con **12 clientes, 16 productos y 84 pedidos** de los últimos 60 días, así los gráficos y reportes tienen información desde el primer momento.

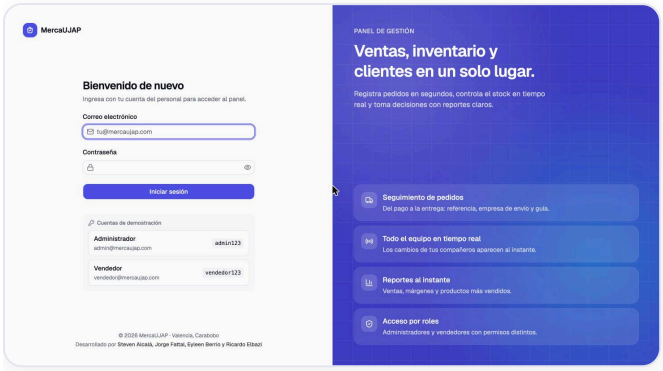
Comandos útiles

```
docker compose ps                       # ver el estado (healthy = listo)
docker compose logs -f servicio-java    # ver los mensajes de un servicio
docker compose down                     # apagar (los datos se conservan)
docker compose down -v                  # apagar y BORRAR los datos (vuelven los de ejemplo)
```

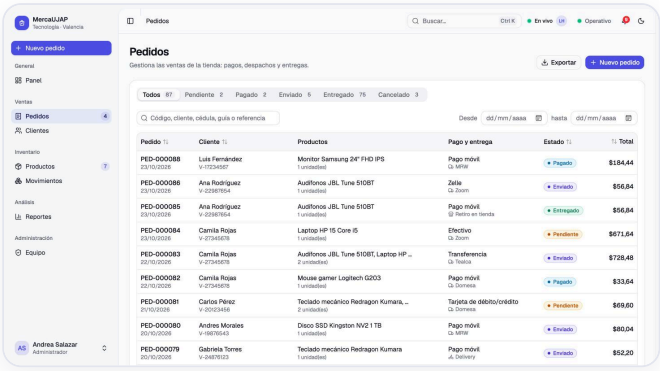
> **¿Ya habías descargado una versión anterior?**
>
> Ejecuta una vez `docker compose down -v` antes de `docker compose up --build`. Si no, la base de datos vieja no tiene las cuentas nuevas y el inicio de sesión falla.

3 Recorrido por el sistema

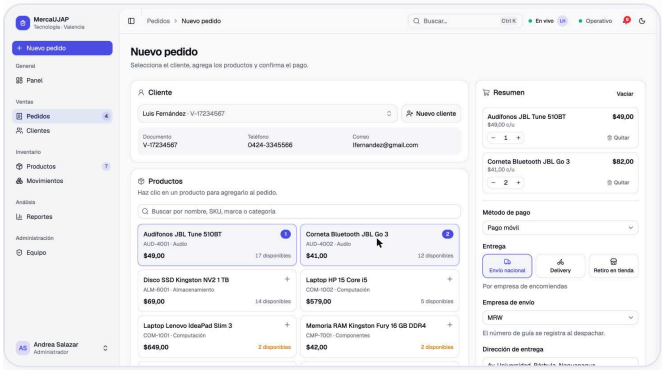
Las pantallas principales y qué se puede hacer en cada una.



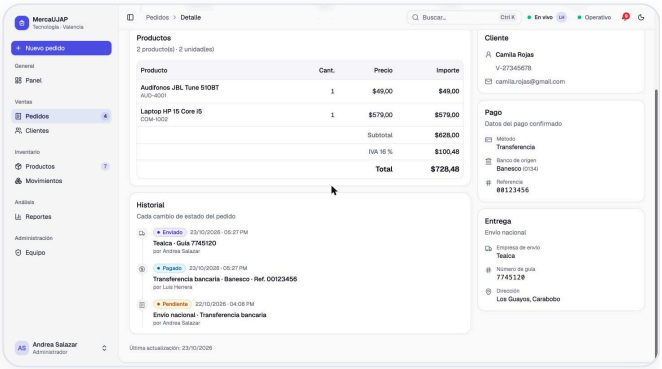
Inicio de sesión. Sin sesión no se puede ver ni hacer nada. Abajo, los integrantes.



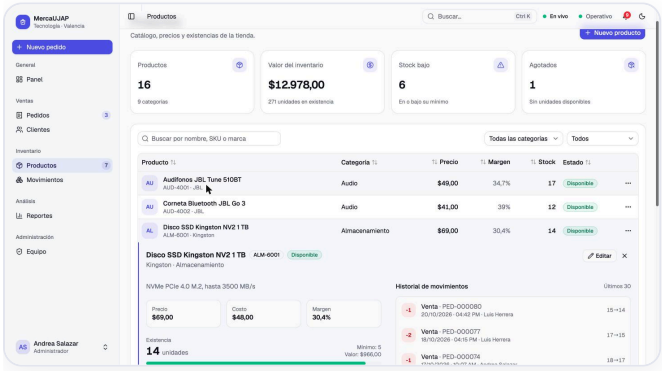
Pedidos. Filtros por estado, búsqueda (también por guía o referencia), fechas y exportación.



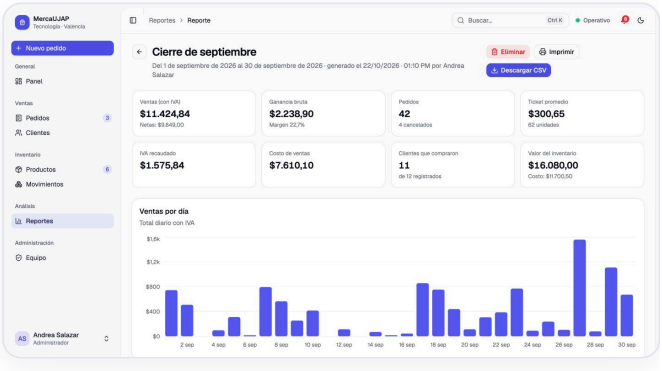
Nuevo pedido. Cliente, productos, método de pago, forma de entrega y empresa de envío.



Detalle del pedido. Datos del pago y de la entrega, e historial de cada cambio.



Productos. El detalle se abre dentro de la tabla: stock, ajuste e historial.



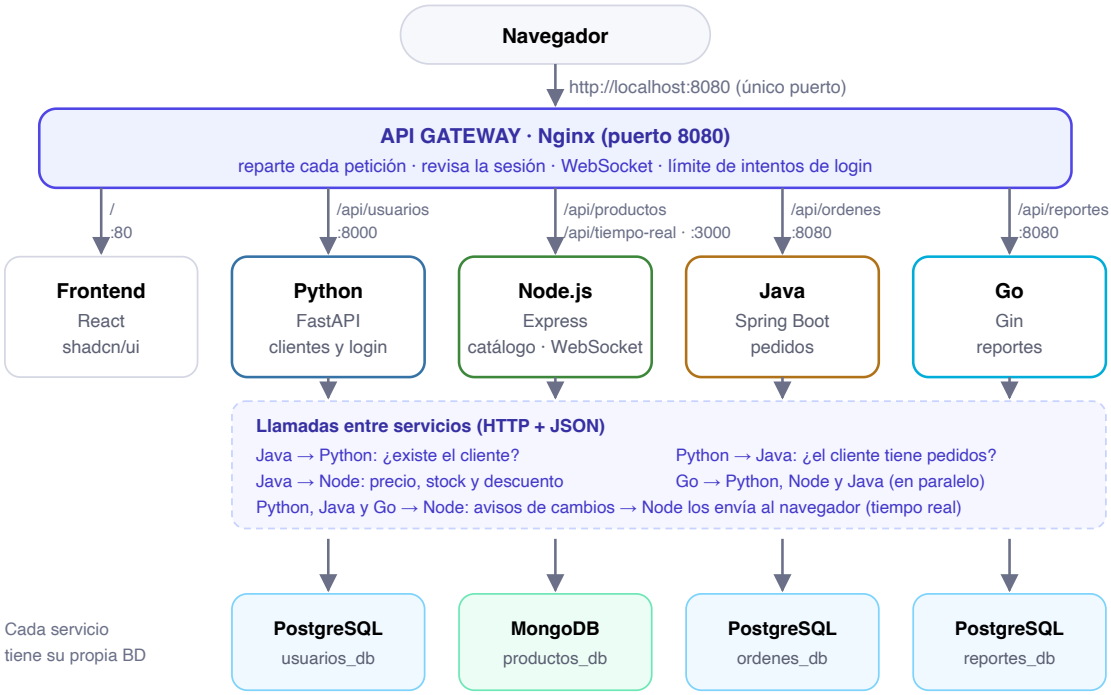
Reportes. Se guardan, se imprimen y se descargan para Excel (CSV).

Otras funciones

- **Clientes** con su historial de compras · **Movimientos** de inventario · **Equipo** (cuentas del personal, solo administrador).
- **Formularios en la misma página:** crear o editar abre el formulario ahí mismo, sin ventanas encima.
- **Buscador global** con `Ctrl + K`, **notificaciones** (alertas de stock y actividad del equipo), indicador **En vivo**, **tema oscuro** y vista para **celular**.

4 Cómo está construido

Son **8 contenedores** de Docker: 4 microservicios, 2 bases de datos, la página web y el gateway. **Solo el gateway abre un puerto en tu PC (8080)**; desde ahí reparte a cada servicio por la red interna.



SERVICIO	LENGUAJE / FRAMEWORK	PUERTO	BASE DE DATOS · ORM	QUÉ HACE
gateway	Nginx	**8080** (único publicado)	—	Entrada única, control de sesión y WebSocket (bonus)
servicio-python	Python · FastAPI	8000 interno	PostgreSQL · SQLAlchemy	Clientes, cuentas del personal, inicio de sesión
servicio-node	Node.js · Express · ws	3000 interno	MongoDB · Mongoose	Catálogo, stock, kardex y canal de tiempo real
servicio-java	Java · Spring Boot	8080 interno	PostgreSQL · Spring Data JPA	Pedidos, estados, pago, envío, historial
servicio-go	Go · Gin	8080 interno	PostgreSQL · GORM	Indicadores, reportes y exportación CSV
frontend	React · Tailwind · shadcn/ui	80 interno	—	La interfaz web

5 El viaje de un pedido

Qué pasa por dentro desde que alguien pulsa **“Registrar pedido”** hasta que aparece en pantalla.

1. **El navegador** envía el pedido a `/api/ordenes` junto con el *token* de sesión.
2. **El gateway (Nginx)** le pregunta a **Python**: “¿este token es válido?”. Si lo es, pasa el pedido a **Java** con el nombre de quien lo registró.
3. **Java le pregunta a Python** si el cliente existe y está activo.
4. **Java le pregunta a Node** el precio y el stock de cada producto. *El precio lo pone el servidor, no el navegador*, así nadie puede hacer trampa cambiándolo.
5. Java calcula subtotal, **IVA 16 %** y total, y **guarda el pedido** en PostgreSQL con su código (por ejemplo `PED-000085`).
6. **Java le pide a Node descontar el stock.** Node lo hace de forma *atómica* (nunca queda negativo) y lo anota como movimiento “Venta PED-000085”.
7. Si algo falla a la mitad (por ejemplo, se acabó el segundo producto), Java **devuelve** el stock que ya había descontado y borra el pedido. Esto se llama **compensación** (patrón *Saga*).
8. La respuesta vuelve al navegador, que abre el detalle del pedido.

Los estados de un pedido

Pendiente → Pagado → Enviado → Entregado

Desde **Pendiente** o **Pagado** se puede pasar a Cancelado, y entonces las unidades vuelven al inventario.
Con **retiro en tienda** se pasa de **Pagado** a **Entregado** sin el paso **Enviado**.

Java **no permite saltos**: si alguien intenta pasar de *Pendiente* a *Entregado* responde con un error. A esto se le llama **máquina de estados**, y está en `EstadoOrden.java`. Cada cambio queda en el historial con fecha, nota y usuario.

Los reportes

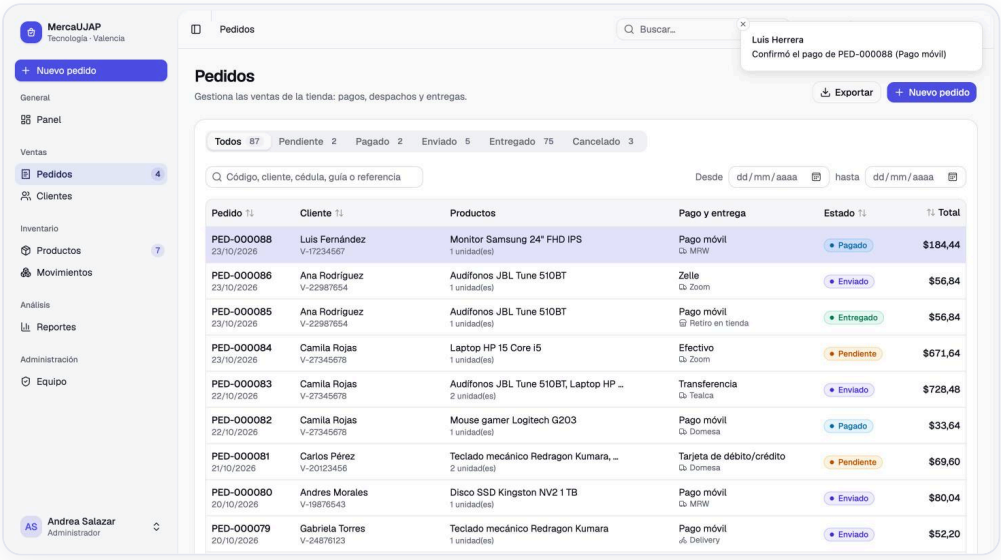
Go le pide los datos a **Python, Node y Java al mismo tiempo** (con *goroutines*) y calcula ventas, ganancia, productos más vendidos, etc. Si uno de los servicios no responde, el reporte **igual se calcula** y el panel avisa cuál faltó.

```
// servicio-go/clientes.go: las 3 consultas se lanzan en paralelo
wg.Add(3)
go consultar("Clientes",  cfg.UsuariosURL+"/usuarios",   &datos.Usuarios)
go consultar("Productos", cfg.ProductosURL+"/productos", &datos.Productos)
go consultar("Pedidos",   cfg.OrdenesURL+"/ordenes",     &datos.Ordenes)
wg.Wait()   // espera a que terminen las 3
```

6 Tiempo real, pago y envío

Si dos personas tienen el panel abierto, lo que hace una **aparece solo** en la pantalla de la otra, sin recargar la página. Esto se hace con un **WebSocket**: una conexión que queda abierta para que el servidor avise al navegador.

1. Al iniciar sesión, el navegador abre **una** conexión a `/api/tiempo-real`. El gateway la valida con el mismo token y la pasa a **Node** (`tiempo-real.js`).
2. Cuando un servicio cambia algo, **avisa a Node**: Python (clientes), Java (pedidos) y Go (reportes) hacen `POST /interno/eventos`. Esa ruta no está en el gateway: solo se usa dentro de Docker.
3. Node reenvía el aviso a **todos los conectados**. Cada pantalla recarga solo lo que cambió, aparece un aviso con el nombre de quien lo hizo y la fila se ilumina. Arriba se ve **quién está conectado**.
4. Si se corta la conexión, el indicador dice *Sin conexión* y se **reconecta solo**.



Luis confirma un pago en su computadora: en la pantalla de Andrea aparece el aviso y la fila del pedido cambia sola a “Pagado”.

Pago y envío: cada dato en su campo

En vez de escribir todo en un texto (“Pago móvil ref. 004512”, “MRW guía 7745120”), cada paso tiene **campos propios** en la base de datos, y Java los valida:

PASO	QUÉ SE REGISTRA	VALIDACIÓN
Pagado	Método, **banco** (0134 Banesco, 0102 Venezuela…) y **número de referencia**	Pago móvil y transferencia: banco obligatorio y referencia **solo con números**
Enviado	**Empresa de envío** (MRW, Zoom, Tealca, Domesa, Liberty Express) y **número de guía**	Obligatorios en envío nacional
Entregado	Quién lo recibió	Opcional
Cancelado	Motivo	Obligatorio

Así se puede **buscar un pedido por su guía o referencia**, y los reportes muestran las ventas por empresa de envío. Los formularios se abren **dentro de la misma página**, sin ventanas encima.

7 Inicio de sesión y permisos

1. En el login, el navegador envía correo y contraseña a **Python**.
2. Python compara la contraseña con su **hash**. Las contraseñas nunca se guardan tal cual, sino una "huella" irreversible (PBKDF2). Si coincide, entrega un **token JWT**: un texto firmado que dice "es Andrea, administradora, válido por 8 horas".
3. El navegador envía ese token en cada petición.
4. **Nginx revisa el token en cada petición** preguntándole a Python antes de dejarla pasar (`auth_request`). Así, ninguno de los otros servicios tiene que preocuparse por la seguridad.

```
# nginx/nginx.conf
location /api/ordenes {
  auth_request /_auth;                # 1) ¿el token es válido? (pregunta a Python)
  proxy_pass http://java_svc/ordenes;   # 2) si lo es, se lo pasa a Java
}
```

ACCIÓN	ADMINISTRADOR	VENDEDOR
Ver todo; crear y editar pedidos, clientes y productos	**Sí**	**Sí**
Eliminar registros	**Sí**	**No**
Crear o editar cuentas del personal (sección Equipo)	**Sí**	**No**

Además, hay un límite de **10 intentos de login por minuto** para frenar a quien intente adivinar contraseñas, y una cuenta desactivada pierde el acceso de inmediato.

8 Conceptos que necesitas saber

CONCEPTO	QUÉ SIGNIFICA
API / endpoint	La forma en que un programa ofrece servicios a otros por la red. Cada "puerta" es un endpoint: `GET /api/productos` = "dame los productos".
Métodos HTTP	**GET** leer · **POST** crear · **PUT** editar · **PATCH** cambiar una parte · **DELETE** borrar (CRUD).
JSON	Formato de texto con el que se mandan datos: `{ "nombre": "María", "rol": "cliente" }`.
Códigos de respuesta	200 OK · 201 creado · 400 datos malos · 401 sin sesión · 403 sin permiso · 404 no existe · 409 conflicto (ej. sin stock) · 503 servicio caído.
Microservicio	Programa pequeño e independiente, con su propia base de datos, que hace una sola tarea.
Docker: imagen y contenedor	La **imagen** es una receta con el programa y todo lo que necesita (se crea con un *Dockerfile*). El **contenedor** es esa imagen funcionando, como una mini computadora aislada.
Docker Compose	Archivo `docker-compose.yml` que describe los 8 contenedores y cómo se conectan.
SQL vs NoSQL	SQL (PostgreSQL) guarda en **tablas** con columnas fijas. NoSQL (MongoDB) guarda **documentos** tipo JSON flexibles.
ORM	Librería para usar la base de datos con objetos del lenguaje en vez de escribir SQL: SQLAlchemy (Python), JPA (Java), GORM (Go), Mongoose (Node).
API Gateway · puerto único	Puerta de entrada única: todo entra por el puerto **8080** y Nginx lo reparte al puerto interno de cada servicio (8000, 3000, 8080…).
WebSocket	Conexión que queda abierta para que el servidor avise al navegador en **tiempo real** (como un chat).
Token JWT	Texto firmado que demuestra que iniciaste sesión; no se puede falsificar.
Variables de entorno (.env)	Contraseñas y configuración fuera del código. El `.env` no se sube a GitHub; se sube `.env.example`.
React · shadcn/ui	React arma la interfaz con **componentes** reutilizables (`<Button>` , `<Card>`). shadcn/ui es una colección de componentes ya diseñados que se copian dentro del proyecto.
Kardex · SKU	Registro de entradas y salidas de inventario · código único de un producto (ej. `COM-1001`).

9 Dónde está cada cosa

```
mercaujap-microservicios/
├── docker-compose.yml      ← levanta los 8 contenedores (solo el gateway abre un puerto)
├── docker-compose.debug.yml← opcional: abre el puerto de cada servicio para depurar
├── README.md · GUIA.md     ← documentación
├── nginx/nginx.conf        ← API Gateway (rutas y control de sesión)
├── database/init.sql       ← crea las 3 bases de datos de PostgreSQL
├── servicio-python/
│   ├── main.py             ← endpoints de /auth y /usuarios
│   ├── seguridad.py        ← hash de contraseñas y tokens JWT
│   ├── models.py           ← tabla "usuarios"
│   └── schemas.py          ← validaciones (email, cédula…)
├── servicio-node/
│   ├── index.js            ← endpoints de /productos y /movimientos
│   ├── tiempo-real.js      ← servidor WebSocket (avisos en tiempo real)
│   ├── producto.model.js   ← cómo es un producto en MongoDB
│   └── movimiento.model.js ← cómo es un movimiento (kardex)
├── servicio-java/src/main/java/com/ujap/ordenes/
│   ├── controller/         ← endpoints REST de /ordenes
│   ├── service/            ← lógica: validar, IVA, stock, compensación
│   ├── model/              ← Orden, estados, métodos de pago, bancos, empresas de envío
│   ├── client/             ← llamadas HTTP a Python y Node
│   └── demo/               ← crea los pedidos de ejemplo
├── servicio-go/
│   ├── main.go             ← rutas de /reportes
│   ├── clientes.go         ← consulta a los 3 servicios en paralelo
│   ├── reportes.go         ← cálculo de indicadores
│   ├── csv.go              ← exportación para Excel
│   └── reportes_test.go    ← pruebas automáticas
└── frontend/src/
    ├── pages/              ← una pantalla por archivo (panel, pedidos, productos…)
    ├── components/ui/      ← componentes de shadcn/ui
    ├── lib/api.js          ← todas las llamadas al gateway
    └── lib/tiempo-real.jsx ← conexión WebSocket y avisos
```

Ver los datos dentro de las bases de datos

```
# PostgreSQL
docker compose exec postgres psql -U admin -d ordenes_db -c "SELECT codigo, usuario_nombre,
estado, total FROM ordenes LIMIT 5;"
# MongoDB
docker compose exec mongodb mongosh productos_db --eval "db.productos.find({}, {sku:1,
nombre:1, stock:1})"
```

Probar la API desde la terminal

```
# Sin sesión, el gateway responde 401
curl -i http://localhost:8080/api/productos
# Swagger de Python: levantar con el archivo de depuración y abrir :8001/docs
docker compose -f docker-compose.yml -f docker-compose.debug.yml up -d
```

10 Guion para la presentación

Un orden recomendado para mostrarlo en 5 a 7 minutos, y la lista de lo que evalúa la rúbrica.

1. **Antes:** levantar el sistema con tiempo. `docker compose ps` → 8 contenedores. **Seguridad y puerto único:** `curl -i http://localhost:8080/api/productos` → **401**. Mostrar en `docker compose ps` que solo el gateway tiene puerto (8080). Iniciar sesión como administrador.
2. **Panel:** explicar los indicadores y cambiar entre 7, 30 y 90 días. *"Estos números los calcula Go consultando a los otros tres servicios al mismo tiempo."*
3. **Nuevo pedido:** elegir cliente, agregar 2 productos y registrar. *"Java le preguntó a Python si el cliente existe y a Node el precio y el stock, y Node descontó las unidades."*
4. En el pedido: **Registrar pago** (banco y solo el número de referencia) → **Marcar como enviado** (empresa y guía). El formulario se abre en la misma página. Mostrar el historial y **Imprimir**.
5. **Productos:** abrir el producto vendido (se despliega dentro de la tabla) → el stock bajó y aparece "Venta · PED-…". Hacer una **Entrada** de 10 unidades.
6. **Tiempo real:** abrir otra ventana (incógnito) como **vendedor**, ponerlas lado a lado y hacer un cambio en una: en la otra aparece el aviso y el dato cambia solo.
7. **Cancelar** otro pedido (el stock vuelve) y **Reportes:** generar el del mes anterior y descargar el CSV.
8. **Roles:** entrar como vendedor → no aparecen "Equipo" ni "Eliminar".
9. **Tolerancia a fallos:** `docker compose stop servicio-node` → el panel avisa que falta el catálogo pero lo demás sigue funcionando. Luego `docker compose start servicio-node`.

Qué pide la rúbrica y dónde mostrarlo

CRITERIO	PTS	CÓMO SE CUMPLE
4 microservicios, uno por lenguaje	6	Python, Node, Java y Go
Dockerfile + docker-compose.yml	4	Un Dockerfile por servicio (y para frontend, gateway y BD) + compose con redes, healthchecks y volúmenes
BD SQL en mínimo 2 servicios	3	PostgreSQL en 3 servicios (Python, Java y Go)
BD NoSQL en mínimo 1 servicio	2	MongoDB en Node
Frontend que consume las APIs	2	React consume los 4 servicios a través del gateway
Comunicación entre microservicios	2	Java → Python y Node · Python → Java · Go → los 3 · todos → Node (tiempo real)
README con diagrama	1	`README.md`
Bonus: API Gateway con Nginx	+2	`nginx/nginx.conf`: puerto único 8080, sesión y WebSocket

11 Preguntas que puede hacer la profesora

¿Por qué microservicios y no un solo programa?

Cada servicio es independiente: se desarrolla y se actualiza por separado, cada uno usa el lenguaje que más le conviene, y si uno falla los demás siguen funcionando (se puede demostrar apagando Node).

¿Cómo se comunican los microservicios?

Por HTTP con JSON (REST), usando el nombre del servicio como dirección dentro de la red de Docker, por ejemplo `http://servicio-node:3000`.

¿Cómo funciona el tiempo real?

Con un WebSocket: el navegador deja una conexión abierta con Node a través del gateway. Python, Java y Go le avisan a Node cada cambio (`POST /interno/eventos`) y Node lo reenvía a todos los navegadores.

¿Por qué un solo puerto? ¿Qué ventaja tiene el gateway?

Es la única puerta: se publica solo el 8080 y Nginx reparte por la red interna. Los servicios y las bases de datos no quedan expuestos, no hay problemas de CORS y la seguridad está en un solo lugar.

¿Por qué el banco y la referencia van en campos separados?

Para validar (la referencia solo acepta números), buscar pedidos por referencia o guía y sacar reportes por empresa de envío. En un texto libre no se podría.

¿Por qué cada servicio tiene su propia base de datos?

Es el patrón *database per service*: ningún servicio toca los datos de otro, sino que se los pide por la API. Por eso el pedido guarda el id del cliente y del producto, más una copia del nombre y el precio del momento.

¿Por qué MongoDB para los productos?

Los productos son documentos con campos flexibles y el historial de movimientos es un registro que solo crece. Ambos encajan bien con documentos JSON.

¿Cómo funciona el inicio de sesión?

Python verifica la contraseña contra su hash y entrega un token JWT firmado. Nginx valida ese token en cada petición preguntándole a Python antes de reenviarla.

¿Qué pasa si en un pedido falla el segundo producto?

Java devuelve el stock que ya había descontado y borra el pedido (compensación, patrón Saga), porque no existe una transacción entre bases de datos distintas.

¿Y si dos personas compran el último producto al mismo tiempo?

Node descuenta con una operación atómica de MongoDB que solo funciona si `stock ≥ cantidad`. Solo una lo logra; la otra recibe “Stock insuficiente”.

¿Qué es la máquina de estados?

La regla que define a qué estados puede pasar un pedido desde el actual. Evita, por ejemplo, entregar un pedido que nunca se pagó.

¿Por qué no se puede borrar un cliente que tiene pedidos?

Porque se perdería la referencia. Python le pregunta a Java cuántos pedidos tiene; si tiene, responde con un error y sugiere desactivarlo.

12 Problemas comunes y soluciones

PROBLEMA	SOLUCIÓN
`Cannot connect to the Docker daemon`	Abre Docker Desktop y espera a que esté en verde.
`port is already allocated` en el puerto 8080	Otro programa usa el 8080. Levanta con otro: `PUERTO=9090 docker compose up --build` (o pon `PUERTO=9090` en el `.env`) y entra a `http://localhost:9090`.
Arriba dice “Sin conexión”	Se cortó el tiempo real (por ejemplo, Node se está reiniciando). Se reconecta solo; si no, recarga la página.
“Correo o contraseña incorrectos” con las cuentas de prueba	Hay una base de datos de una versión anterior: `docker compose down -v` y luego `docker compose up --build`.
La página dice “Datos incompletos” o no carga datos	Algún servicio sigue arrancando (Java tarda unos 30 s). Espera y recarga. Revisa con `docker compose ps`.
Cambié el código y no se ve el cambio	Vuelve a ejecutar con `--build`.
Quiero volver a los datos de ejemplo	`docker compose down -v` y `docker compose up --build`.
Windows: error con WSL 2	En PowerShell como administrador: `wsl --update`, y reinicia.
Tarda mucho la primera vez	Es normal: se descargan las dependencias de Java, Go y React.

¿Quieres profundizar?

El repositorio tiene el `README.md` con la lista completa de endpoints y el archivo `GUIA.md` con más detalle de cada archivo. Todo el código tiene comentarios en español.

github.com/stvnalc/mercaujap-microservicios